

Using a Large Language Model (LLM) to build a Knowledge Graph from scratch to improve Retrieval-Augmented Generation (RAG) for answering questions on custom data

Bachelor Thesis Presentation | Faculty of Computer Science | University of Vienna

Dragomir Marinov

Table of Contents

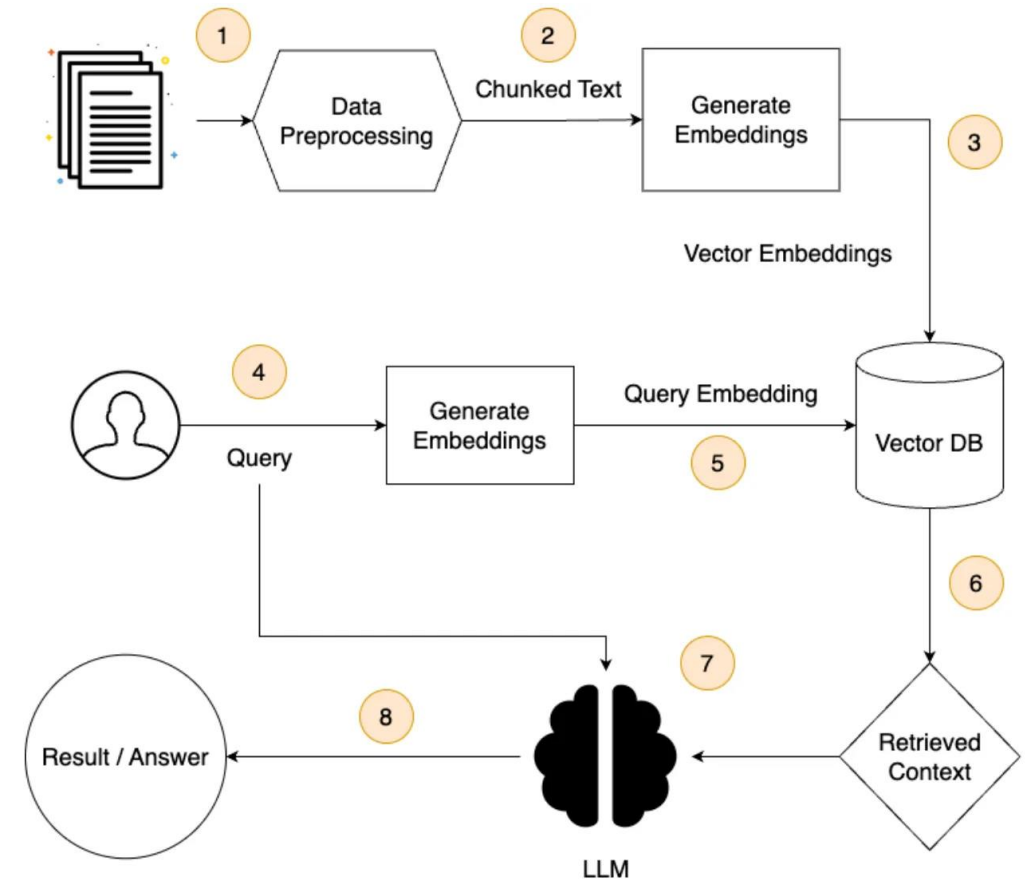
1. Introduction
2. Motivation and Problem Statement
3. Project Goals
4. Tech Stack / Used Technologies
5. Workflow
6. Knowledge Graph Workflow
7. Chatbot Workflow
8. Knowledge Graph Integration
9. Results
10. Demo

Introduction

- Knowledge graph framework that extract entities and relationships from user-provided documents.
- User can view and edit the extracted graph - CRUD
- Knowledge Graph is integrated into a RAG chatbot to improve answer quality.

Motivation: What is RAG?

- LLMs are trained on general internet data - they do not know private documents
- RAG^[1] enables question answering on any custom domain without retraining
- Reduces hallucination - the model references text rather than reconstructing from training weights^[1]
- Rag Flow:
 - split into chunks → embed → store in vector DB →
 - fetch relevant chunks at query time → inject into prompt → LLM answers grounded in the data



Graph taken from: ^[10]

Problems with RAG

- **Chunk homogeneity** - retrieved chunks are semantically similar, giving the same information rephrased rather than diverse facts. [2]
- **No relational context** - chunks are isolated, RAG cannot connect them unless they are retrieved together. [3]
- **Hallucination persists** - when retrieved context is incomplete, the model fills gaps with invented facts. [4] [2]

Improvement with Knowledge Graph

KG stores entities + relationships in a structure

- ① → Structured retrieval gives diverse, non-redundant relational facts [6]
- ② → Relationships between entities are stored and retrievable [5]
- ③ → Hallucinations decrease when responses are based on verified graph facts [3]

Project Goals

- **Primary goal:** Build a Knowledge Graph framework that extracts entities and relationships from provided documents using a local LLM.
 - Additionally:
 - Extract description for the nodes
 - Store in a file system or database
 - CRUD operations for nodes and relationships
- **Secondary goal:** Integrate the Knowledge Graph into a RAG chatbot to improve answer quality.
 - Specific tasks:
 - Create simple RAG Chatbot
 - Choose integration strategy
 - Evaluate Results

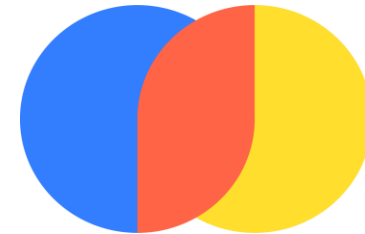
Tech Stack

- Backend: **Python 3.11**
 - Suited for AI tasks
- UI: **Streamlit**
 - Python native
 - Built-in file uploaders, chat components
 - *streamlit-agraph* library for graph visualization
- LLM: **llama3.1:8B**
 - Small model, working on most machines
 - Free, local

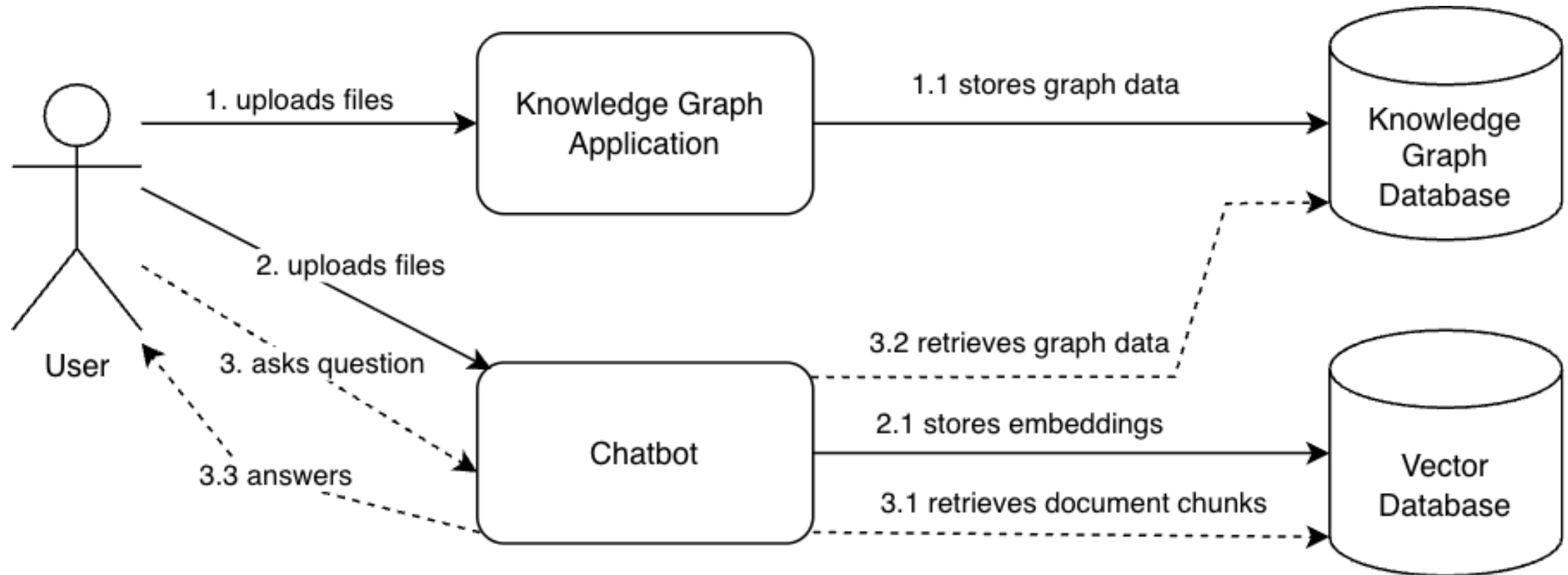


Databases

- Knowledge Graph: **mongodb**
 - No fixed schema
 - Relationship structure maps naturally to flexible documents
- Chatbot: **chromadb**
 - Fully embedded, in-process
 - Persists on disk



Workflow

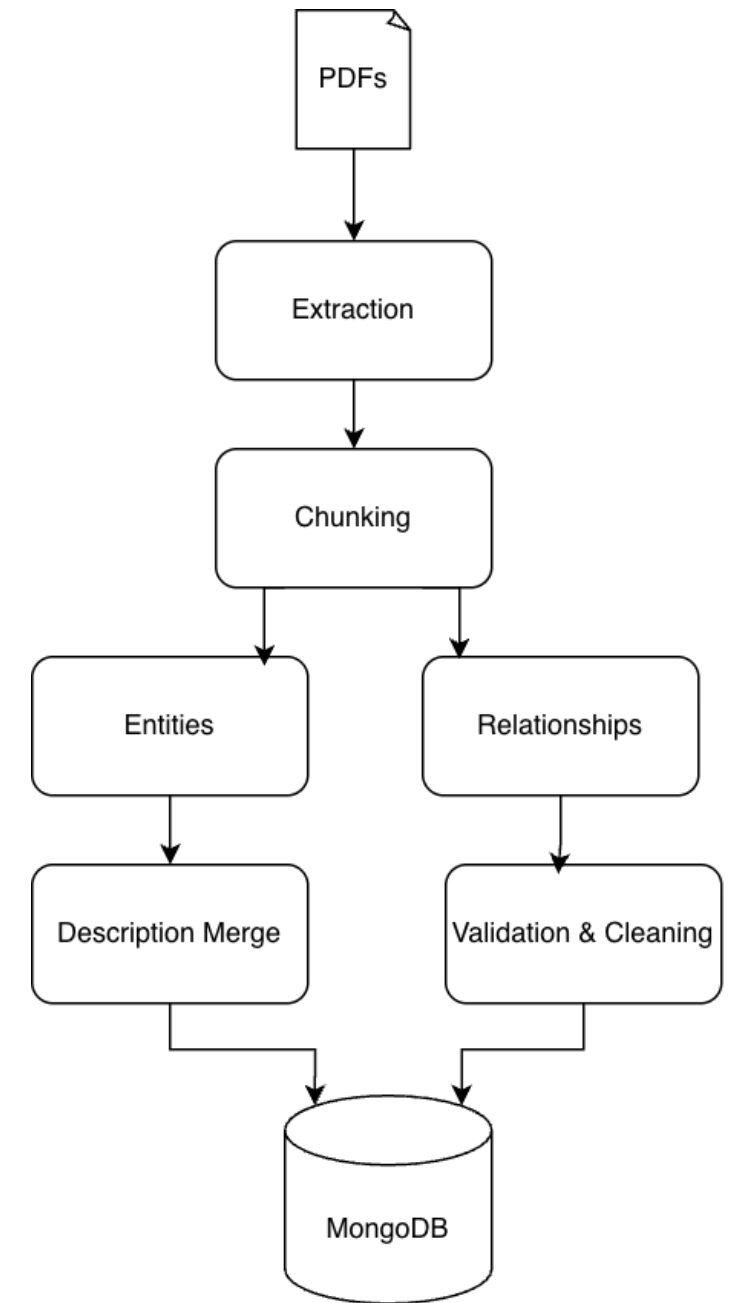


- Ingestion phase – documents go into both apps

- Query phase – chatbot retrieves from both databases

Knowledge graph Workflow

- Text extraction (PDFs - PyPDFLoader, URLs - tags parsing)
- Chunking - LangChain text splitter
- LLM extracts entities + relationships simultaneously
- Duplicate entities merged - descriptions combined
- Relationships validated - malformed relationships removed
- Stored in MongoDB



MongoDB Data Structure

Entities collection:

one document per named entity, with a name and a merged description

```
{  
  "name": "Process Mining",  
  "description": "A discipline at the intersection of data science  
                 and process science, used to analyze event logs  
                 and discover, conform, or enhance process models."  
}
```

Relationships collection:

subject-predicate-object triples as the standard Knowledge Graph representation

```
{  
  "subject": "CustPro Project",  
  "predicate": "applies",  
  "object": "Process Mining"  
}
```

Integration Strategy

Strategy	How it works	Limitations
Entity-aware Chunk Retrieval [9]	Link chunks to KG entities for re-ranking	Adds re-ranking complex without clear quality gain over direct injection
KG Query Rewriting [9]	Expand the user query using KG before vector search	KG facts never reach the LLM - only improves retrieval quality
KG Embeddings in Vector DB [9]	Convert KG nodes/edges to text, embed them alongside chunks	Loses explicit graph structure - KG treated as flat text
KG-Guided Post-RAG Refinement [2]	RAG generates answer first, then KG validates in second LLM call	Two LLM passes - doubles latency, impractical with local 8B model

KG-Augmented Retrieval [11]	Query KG + vector DB and combine results into one context	

Integration Strategy - My Approach

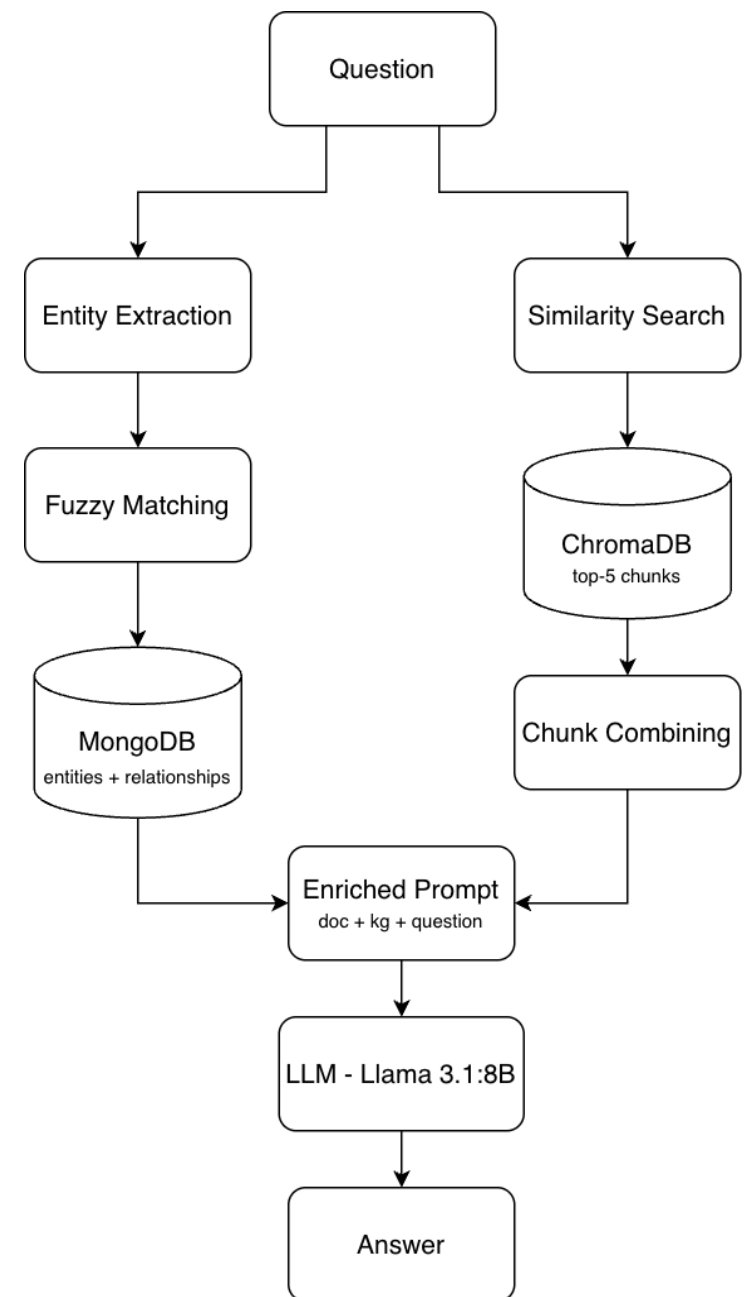
Chosen strategy: **KG-Augmented Retrieval**

Two steps in the pipeline:

- **Hybrid Retrieval** - *what to retrieve?* (inspired by HybridRAG [11])
 - Vector search → top-5 chunks
 - Knowledge Graph lookup → entities + relationships
- **Prompt Enrichment** - *how to use it?* (inspired by KG-augmented prompting [8])
 - Both contexts injected into a single prompt
 - Document context = primary source; KG context = supplementary relational information
 - Single LLM inference call

Chatbot Workflow

- Two retrievals:
 - Vector search
 - Knowledge Graph entity lookup
- Fuzzy matching connects query terms to Knowledge Graph entity names
- Single enriched prompt -> single LLM inference call -> answer



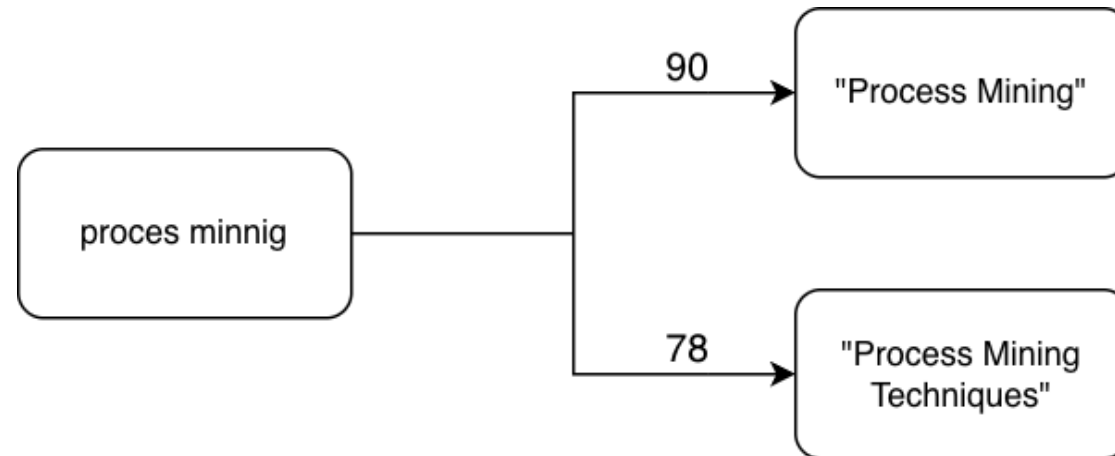
Fuzzy Matching

- Problem with string match:



- Fuzzy Matching:

- Confidence threshold of 75
 - below 75: unrelated entities matched
 - above 75: valid variations rejected
- Number of matched entities: 2



Evaluation - Custom Tool

- Llama 3.1:8B-as-Judge
- Criteria (0-3 scale):
 - *Correctness* - Are the stated facts accurate?
 - *Faithfulness* - Are all claims grounded in the retrieved context?
 - *Completeness* - How many key points from the ground truth does the answer cover?
 - *Relevancy* - Does the answer address the question asked?
 - *Conciseness* - Is the answer focused, without unnecessary noise?

FINAL COMPARISON – RAG only vs KG + RAG				
Criterion	RAG only	KG + RAG	Δ	
Correctness	2.20	2.40	▲ 0.20	
Faithfulness	1.80	2.20	▲ 0.40	
Completeness	2.60	2.20	▼ 0.40	
Relevancy	2.60	2.80	▲ 0.20	
Conciseness	1.60	2.00	▲ 0.40	
OVERALL	2.16	2.32	▲ 0.16	(max 3.0)

Evaluation - Custom Tool

- Problems with:
 - *Completeness* - Ground truths as sentences → rewritten as **numbered key point checklists**
 - *Relevancy* - scored near-maximum for both pipelines → replaced with **Specificity**
 - *Conciseness* - penalized KG+RAG for citing two sources → **removed**
 - Integer scale caused rounding noise → changed to **0.00 - 1.00 decimal**
- Root cause:
 - Llama 3.1:8B too small to judge reliably - inconsistent scores between runs, format errors, criteria conflation

Evaluation - External LLM

- Changes:
 - Use of bigger external model (Claude Sonnet) as a judge
 - Source PDFs attached for direct verification against documents
 - Questions separated in different types:
 - factual, multi-fact, relational, causal
- Limitations:
 - For single short document
 - chunk retrieval already performs well
 - limited room for KG to outperform

Summary — weighted averages across all 10 questions				
Criterion	RAG only	KG+RAG	Δ	
Correctness (×0.30)	0.800	0.900	+0.100	
Completeness (×0.25/0.30)	0.638	0.750	+0.113	
Faithfulness (×0.30)	0.825	0.950	+0.125	
Specificity (×0.15/0.10)	0.663	0.725	+0.063	
Weighted overall avg	0.736	0.841	+0.105	

Q-level weighted totals at a glance				
Q	Type	RAG only	KG+RAG	Δ
1	factual	0.538	0.763	+0.225
2	factual	0.251	0.575	+0.326
3	factual	0.663	0.588	-0.075
4	multi	0.676	0.963	+0.288
5	multi	1.000	1.000	0.00
6	relational	0.800	0.900	+0.100
7	relational	0.725	0.900	+0.175
8	relational	0.975	0.975	0.00
9	causal	0.900	0.900	0.00
10	causal	0.830	0.855	+0.025

Evaluation - Cross-Document Questions

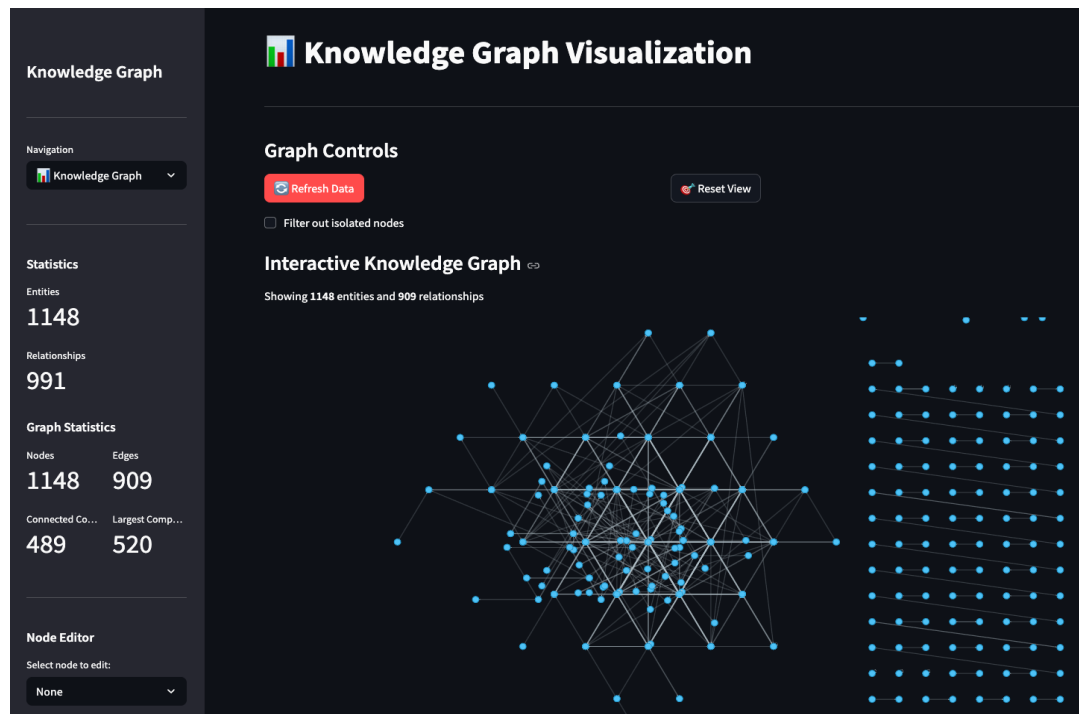
- Changes:
 - 2 more documents added
 - Questions extended to 19
 - Added **cross-document category**
- Results:
 - KG+RAG wins all 6 cross-document questions
 - avg improvement of **+0.146**

Cross-doc bridge analysis — did KG help connect papers?

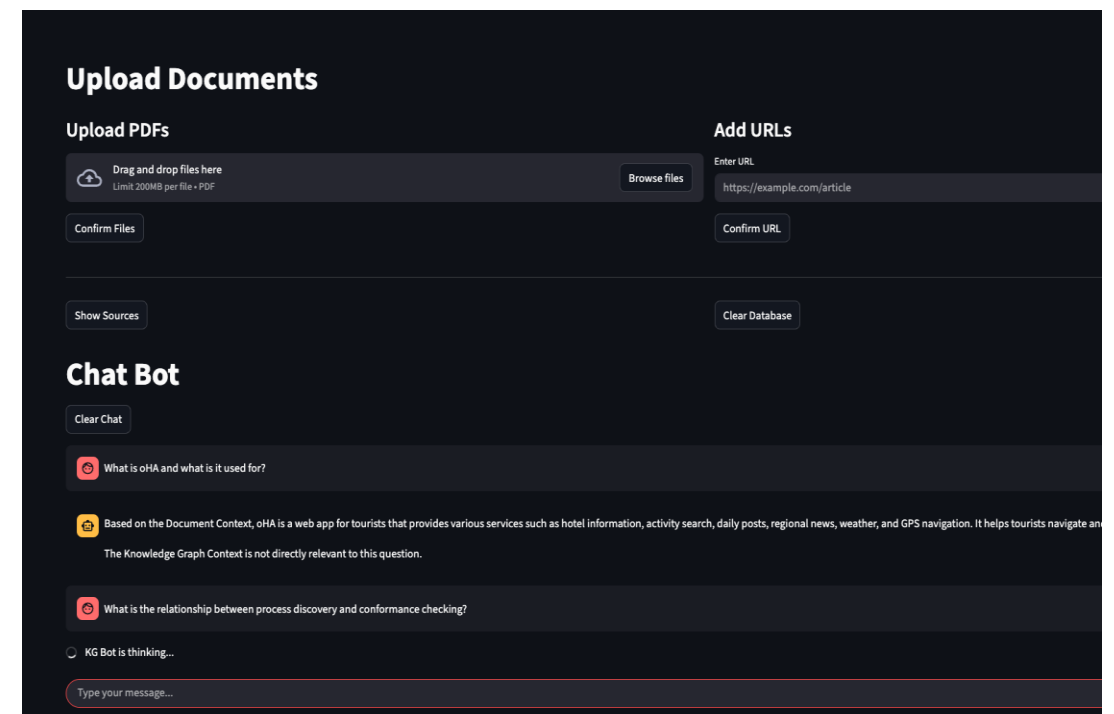
Q	Papers linked	RAG only	KG+RAG	Bridge quality
Q12	oHA + vdA	0.238	0.776	Strong — B correctly maps all 3 PM types to oHA activities with structured table
Q15	oHA + Koschmider	0.603	0.698	Moderate — B connects abstraction step to micro services correctly
Q16	oHA + vdA	0.603	0.698	Moderate — B links scattered data problem to oHA solution more explicitly
Q13	oHA + Koschmider	0.213	0.348	Weak — both fail; B marginally better by not falsely denying oHA access
Q14	oHA + vdA + Koschmider	0.288	0.333	Weak — both miss GT algorithms; B slightly better by attempting names
Q17	oHA + vdA	0.308	0.358	Weak — both fail the OCEL↔multi-level case connection; B marginally better

Demo

Knowledge Graph



Chatbot



The Chatbot interface has a dark theme. It includes an 'Upload Documents' section with a 'Drag and drop files here' area (Limit 200MB per file • PDF) and a 'Browse files' button. Below this is a 'Confirm Files' button. To the right, the 'Add URLs' section has an 'Enter URL' field with the example 'https://example.com/article' and a 'Confirm URL' button. Below these are 'Show Sources' and 'Clear Database' buttons. The 'Chat Bot' section features a 'Clear Chat' button and a chat history with three messages: 'What is oHA and what is it used for?', 'Based on the Document Context, oHA is a web app for tourists that provides various services such as hotel information, activity search, daily posts, regional news, weather, and GPS navigation. It helps tourists navigate and... The Knowledge Graph Context is not directly relevant to this question.', and 'What is the relationship between process discovery and conformance checking?'. A status indicator shows 'KG Bot is thinking...'. At the bottom is a 'Type your message...' input field.

Summary

- What was built:
 - Knowledge Graph Application - automatically extracts entities with descriptions and relationships from any user-provided documents; fully local, domain-agnostic
 - RAG Chatbot - dual retrieval from vector DB and knowledge graph, combined through a single enriched prompt into one LLM inference call
- Future work:
 - Better entity linking to reduce cross-contamination noise
 - Larger local models (13B+) to better exploit KG context
 - Query-type routing - activate KG only for queries likely to benefit from cross-document relational reasoning

Lessons Learned

- RAG is already good for factual queries, the knowledge graph improves multi-document reasoning
- Entity linking precision is a critical parameter - imprecise matching introduces noise on simple questions
- Small local models (8B) have limited capacity to extract entities and interact with structured context - larger models would show better results
- Evaluation methodology matters - a good judge is needed for trustworthy results

References

- [1] Lewis et al. (2020). RAG for Knowledge-Intensive NLP. arXiv:2005.11401
- [2] Gao et al. (2024). RAG Survey. arXiv:2312.10997
- [3] Pan et al. (2024). Unifying LLMs and KGs. arXiv:2306.08302
- [4] Tonmoy et al. (2024). Hallucination Mitigation Survey. arXiv:2401.01313
- [5] Ji et al. (2021). KG Survey. IEEE TNLS 33(2):494–514
- [6] Zhu et al. (2025). KG-Guided RAG. arXiv:2502.06864
- [7] Soman et al. (2023). KG-RAG. arXiv:2405.12035
- [8] Baek et al. (2023). KAPING. arXiv:2306.04136
- [9] Peng et al., 2024, ACM Transactions on Information Systems, arXiv:2408.08921
- [10] Jain, T. Evaluate RAG pipeline using HuggingFace open source models. Hugging Face Community Blog, June 2024. Accessed 2025.
- [11] Sarmah et al. (2024), arXiv:2408.04948